

User Interface Layer

User Interface Layer

A **user interface (UI)** is a channel between human and computer interaction, where a user interacts with and controls a computer or machine to complete tasks effectively and efficiently (Everyinteraction.com, n.d.). It determines how commands are given to a computer or a program and how information is going to be displayed (Businessdictionary.com, n.d.). It is also a set of software that operates above the core operating system but under the application layer. Thus, the UI layer is largely composed of APIs and a window manager.

An **application programming interface (API)** is a set of routines, protocols, and tools for building software applications, especially when programming a graphical user interface (GUI). It also specifies how software components should interact (Webopedia.com, n.d.).

When an application in a device is used, it connects to the Internet and sends data to a server. The server then retrieves that data, interprets the data, performs necessary actions, and sends the data back to the device. Afterward, the application interprets and presents the data to the user in a readable way. All these processes happen through APIs (MuleSoft, n.d.).

A **window manager** is a software utility found in most GUI-based software and applications that manage the overall alignment and layout of graphical windows. It defines and controls the appearance and positioning of the application interface of windows (Techopedia, n.d.).

In most cases, Techopedia argues, window managers work in collaboration with the desktop environment, the logical graphical system, and the underlying hardware to operate and manipulate graphical windows. It provides the ability of a software or program to open, minimize, maximize, and close windows. The placement, effects, colors, and transitioning may also be part of the window manager, depending on the operating system, graphical libraries, and system in use.

Characteristics of a successful UI

- It should be intuitive, which does not necessarily require training to operate.
- It should be efficient to carry out a smooth operation for faster usage.
- It should be user-friendly to provide user satisfaction.

Types of User Interface (UI)

- **Natural Language Interface** – This requires the user to enter responses to questions asked by the computer. The questions are displayed on a virtual device unit (VDU), commonly a monitor, and the answer are entered via the keyboard. It is called “natural language” interface because the computer and the user seem to have a conversation as their mode of interaction (theteacher.info Ltd., n.d.).
- **Menu-Based Interface** – This provides the user with an on-screen list of available selections, although the options displayed are limited. In this matter, the user need not to know the system, but the tasks that should be accomplished. Menu-based interfaces differ depending on the specs of the hardware. These can also be nested within one another to lead the user through the options in a program or application (W3Computing, n.d.).
- **Form-Based Interface** – It consists of on-screen forms or Web-based forms displaying fields containing data items or parameters that need to be communicated to or solicited from the user. The form often is a facsimile of a paper form. Form-based interfaces are set up to show what information should be inputted and where (W3Computing, n.d.).
- **Command Line Interface (CLI)** – It allows the user to control the application with a series of keystrokes, commands, phrases, or some sequence of these three (3) methods, though the command language has

no inherent meaning for the user. This interface requires memorization of syntax (W3Computing, n.d.). It can take a long time to learn, but it is not intuitive. On the other hand, experienced users tend to prefer the command-line interface because of the faster completion time (theteacher.info Ltd., n.d.).

- **Graphical User Interface (GUI)** – Most interfaces today are already graphical. This type of interface is expected to be available in a multi-tasking environment or an application that involves a considerable degree of complexity (theteacher.info Ltd., n.d.). The creation of GUIs poses a challenge because an appropriate model or acceptable conceptual model of representation must be invented. Thus, designing GUIs for use on the Internet, extranets, and Web requires more careful planning (W3Computing, n.d.).

Input and Output

Input pertains to any information or data that is sent to a computer for processing. Input or user input is directed to a computer using an *input device* like keyboard, mouse, and microphone. On the other hand, **output** refers to any information that was processed by and sent out from a computer or any electronic device. Anything that can be viewed on a screen, such as typed words and images, can be considered as an output (Computer Hope, n.d.).

According to Kim (2015), inputs and outputs (I/O) are handled by the interrupt mechanism of an operating system (OS) at a low-level operation. An **interrupt** is a signal to the processor indicating that an I/O event has occurred and must be handled. An interrupt signal is interpreted so that the address of its handler procedure can be looked up and executed while suspending the ongoing process for a moment. After finishing the handler procedure, the suspended process resumes again. An arrival of an interrupt is checked at a very fast rate as part of the processor execution cycle. This means that the processor is always listening to the incoming events, ready to serve as needed.

At a higher level, input and output operations are frequently described in terms of *events* and *event handler*, which are based on the idea of lower-level interrupt mechanism. Generally, this is called an **event-driven program structure** in which programs are developed in terms of events, such as mouse clicks, gestures and keyboard input, and their corresponding handlers.

Events

Event-driven program structure is a programming paradigm in which the flow of program execution is determined by events like a mouse click, keypress, or a message from the operating system or another program. It can be written in any programming language (e.g., Visual Basic, Visual C++, and Java) which is object-oriented and has a visual language that supports event-driven programming (TechnologyUK, 2018). An event-driven application is designed to detect events as they occur and then deals with them using an appropriate event-handling procedure.

Below are the components and common event classes in Java according to Studytonight and ThoughtCo:

Parts of Event-Driven Programming

- **Event** – This object in Java is created when something changes within a GUI. An event is triggered, and a relevant event object is created when a user clicks on a button, clicks on a combo box, types characters into a text field, etc. This behavior is part of Java's event handling mechanism and is included in the Swing GUI library, which is supported by several Java packages like *java.util*, *java.awt*, and *java.awt.event*.
- **Event Source** – It refers to the object that is triggered in an event. In other words, even the source is an object that generates an event.
- **Event Listener** – It is a program code that listens for changes, additions, user interaction, etc. When an event listener is performed, it does the specified task(s) based on the event.

In handling events, a source generates an event and sends it to one (1) or more listeners registered with the source. Once the listener receives the event, it processes and returns the event.

With this, most user interface frameworks operate similarly with an event-driven structure. The hardware input devices generate events that are channeled to the software interface, which is composed of UI objects, and are processed to produce output by the event-handling codes. The user interface layer laid out above the OS provides the computational framework for an event-driven processing model. Then, it performs abstractions of the lower OS details for easier and intuitive, interactive software and interface (Kim, 2015).

Event classes represent different core event handling mechanism in Java. Below are the common event classes used in developing an event-driven program.

Event Classes	Event Description
ActionEvent	This occurs when a graphical element is clicked, such as a button or an item in a list.
ContainerEvent	This represents an event that occurs to the GUI's container itself—for example, a user adds or removes an object from the interface.
KeyEvent	This occurs when the user presses, types, or releases a key in the keyboard.
WindowEvent	This represents an event relating to a window—for example, when a window is activated and closed.
MouseEvent	This represents any related mouse action, such as clicking, pressing, and dragging.
TextEvent	This is generated when the value of <i>textarea</i> or <i>textfield</i> is changed.
ComponentEvent	This is generated when the component is hidden, moved, or resized.
AdjustmentEvent	This is generated when the scroll bar is used.

Table 1. List of common event classes in Java with its corresponding description.

In an event-driven program, it is also necessary to know that multiple listeners and event sources can interact with one another. Multiple events can be registered by a single listener if they are of the same type. Meaning, for a similar set of components that perform the same type of action, one (1) event listener can handle several events. Similarly, although not common, a single event can be bound to multiple listeners if it suits the program design.

References:

- Beal, V. (n.d.). API – application program interface. In *Webopedia*. Retrieved from <https://www.webopedia.com/TERM/A/API.html> on July 5, 2019
- Event-driven Programming. (n.d.). In *TechnologyUK*. Retrieved from <http://www.technologyuk.net/software-development/designing-software/event-driven-programming.shtml> on July 9, 2019
- Gibbs, M. (n.d.). Event-Driven Programming in Java: Definition & Components. Retrieved from <https://study.com/academy/lesson/event-driven-programming-in-java-definition-components.html> July 9, 2019
- Graphic. (n.d.). In *Dictionary.com*. Retrieved from <https://www.dictionary.com/browse/graphical> on July 5, 2019
- Input. (2019, June 30). In *Computer Hope*. Retrieved from <https://www.computerhope.com/jargon/i/input.htm> on July 8, 2019
- Kim, G. (2015). *Human-computer interaction fundamentals and practice*. USA: CRC Press.
- Leahy, P. (2019, July 3). A Java Event Represents a GUI Action in Java's Swing GUI API. In *ThoughtCo*. Retrieved from <https://www.thoughtco.com/event-2034091> July 9, 2019
- Output. (2019, June 30). In *Computer Hope*. Retrieved from <https://www.computerhope.com/jargon/o/output.htm> on July 8, 2019
- Types of user interface. (2016, September). In *theteacher.info Ltd*. Retrieved from <http://theteacher.info/index.php/systems-software/notes/4623-types-of-user-interface> on July 5, 2019
- Types of User Interface. (n.d.). In *W3Computing*. Retrieved from <https://www.w3computing.com/systemsanalysis/types-user-interface/> on July 5, 2019
- User interface. (n.d.). In *Business Dictionary*. Retrieved from <http://www.businessdictionary.com/definition/user-interface.html> on June 28, 2019
- User Interface. (n.d.). In *Every Interaction*. Retrieved from <https://www.everyinteraction.com/definition/user-interface/> on June 28, 2019
- User interface (UI). (2016, November). In *TechTarget*. Retrieved from <https://searchmicroservices.techtarget.com/definition/user-interface-UI> on June 28, 2019
- What is an API? (Application Programming Interface). (n.d.). In *MuleSoft*. Retrieved from <https://www.mulesoft.com/resources/api/what-is-an-api> on June 28, 2019
- Window Manager. (n.d.). In *Techopedia*. Retrieved from <https://www.techopedia.com/definition/10043/window-manager> on July 5, 2019